This doesn't resolve all the problems, though. There's another problem: if two threads read the same value of `head`, they'll return the same node. This violates the intent of the stack data structure, so you need to avoid this. You can resolve this the same way you resolved the race in `push()`: use compare/exchange to update `head`. If the compare/exchange fails, either a new node has been pushed on or another thread just popped the node you were trying to pop. Either way, you need to return to step 1 (although the compare/exchange call rereads `head` for you).

Once the compare/exchange call succeeds, you know you're the only thread that's popping the given node off the stack, so you can safely execute step 4. Here's a first cut at `pop()`:

```
template<typename T>
class lock_free_stack
{
public:
    void pop(T& result)
    {
        node* old_head=head.load();
        while(!head.compare_exchange_weak(old_head,old_head-
>next));
        result=old_head->data;
    }
};
```

Although this is nice and succinct, there are still a couple of problems aside from the leaking node. First, it doesn't work on an empty list: if `head` is a null pointer, it will cause undefined behavior as it tries to read the `next` pointer. This is easily fixed by checking for `nullptr` in the `while` loop and either throwing an exception on an empty stack or returning a `bool` to indicate success or failure.

The second problem is an exception-safety issue. When we first introduced the thread-safe stack back in chapter 3, you saw how just returning the object by value left you with an exception safety issue: if an exception is thrown when copying the return value, the value is lost. In that case, passing in a reference to the result was an acceptable solution because you could ensure that the stack was left unchanged if an exception was thrown. Unfortunately, here you don't have that luxury; you can only safely copy